Práctica 9 – Sobrecarga de Métodos y Operadores en C++

Nombre del alumno: cesar zaid reynoso Martinez

Materia: Programación Orientada a Objetos

Docente: jesus collazo

Fecha: 15/04/2026

Institución:instituto tecnologico superior de rioverde

Introducción

En esta práctica se analizó un programa en C++ que implementa el uso de sobrecarga de métodos
y operadores. El objetivo fue comprender cómo un mismo método puede comportarse de distintas
formas dependiendo de los parámetros, así como entender cómo los operadores pueden redefinirse para trabajar con objetos.

1. Arquitectura del Proyecto

El proyecto sigue el principio de **encapsulamiento**, dividiendo la lógica en tres componentes esenciales para facilitar el mantenimiento y la escalabilidad.

A. Archivo de Cabecera (`Pacman.h`)

Este archivo actúa como el "contrato" de la clase. Define la estructura de los datos y las firmas de los métodos que estarán disponibles.

> **Atributos Privados:** Se definen variables como `int puntos`, `int x`, `int y` (coordenadas) y `string nombre`. Al ser privados, protegemos la integridad de los datos.
>
> **Prototipos de Métodos:** Se declaran los constructores (por defecto y parametrizados) y las funciones miembro.
>
> **Declaración de Operadores:** Se definen los prototipos para la sobrecarga de operadores que permitirán manipular el puntaje de forma intuitiva.

B. Implementación (`Pacman.cpp`)

Aquí se desarrolla la lógica de negocio. Se utiliza el operador de resolución de ámbito `::` para vincular las funciones a la clase `Pacman`.

C. Programa Principal (`main.cpp`)

Es el punto de entrada. Se encarga de instanciar los objetos de la clase `Pacman`, gestionar el bucle de juego (si aplica) y realizar las llamadas a los métodos para demostrar la funcionalidad del sistema.

2. Sobrecarga de Métodos: Polimorfismo en Movimiento

La sobrecarga de métodos permite que una misma acción (mover) se comporte de manera diferente según el contexto de los argumentos enviados.

mover() (Sin parámetros):

Implementa un movimiento por defecto. Por ejemplo, Pacman sigue avanzando en la última dirección registrada o se mueve una unidad hacia adelante.

mover(char direccion):

Permite el control direccional. Utiliza una estructura switch para evaluar los caracteres ('W', 'A', 'S', 'D' o 'U', 'D', 'L', 'R') y actualizar las coordenadas x o y correspondientes.

mover(int pasos):

Ideal para movimientos automáticos o "power-ups" donde el personaje se desplaza una distancia fija en la dirección actual.

3. Sobrecarga de Operadores: Manipulación de Puntaje

Para que el código sea más legible y "limpio", se han sobrecargado los operadores aritméticos. Esto permite tratar al objeto Pacman casi como si fuera un tipo de dato numérico cuando se trata de sus puntos.

Operador Binario +

Permite sumar puntos de un evento externo y devolver un nuevo valor o un objeto actualizado.

Puntos_{final} = Puntos_{actual} + N

Operador de Asignación Compuesta `+=`

Es fundamental para la recolección de "pellets" o frutas. En lugar de escribir `pacman.setPuntos(pacman.getPuntos() + 10)`, la sobrecarga permite una sintaxis mucho más elegante:

```
pacman += 10;
```

> **Nota técnica:** El método operator+= generalmente devuelve una referencia al objeto actual (*this) para permitir el encadenamiento de operaciones.

4. Flujo de Ejecución Detallado

Instanciación: Se crea el objeto `Pacman miPacman("Jugador1");` inicializando los puntos en cero y las coordenadas en el origen.

Interacción: El usuario ingresa un comando. Si presiona una tecla de dirección, se invoca `mover(direccion)`.

Actualización de Estado: Si el personaje colisiona con un ítem, el programa ejecuta `miPacman += valorFruta;`.

Salida: El método `mostrarEstado()` (comúnmente implementado) imprime en consola la posición actual (x, y) y el total de puntos acumulados.

Estructura Sugerida del Código (Resumen)

Archivo	Responsabilidad Principal
Pacman.h	Definición de la clase, atributos (puntos, posición) y firmas de métodos.
Pacman.cpp	Lógica de switch para movimientos y aritmética de la sobrecarga de +.

Archivo	Responsabilidad Principal
main.cpp	Creación del objeto, flujo de control y pruebas de los métodos sobrecargados.

Análisis

La sobrecarga de métodos permite definir múltiples funciones con el mismo nombre pero diferente
lista de parámetros. El compilador decide cuál función ejecutar dependiendo de los argumentos
utilizados en la llamada.
Por ejemplo, si se llama mover() sin parámetros, se ejecuta una versión diferente a cuando se
llama mover('w') o mover(5). Esto facilita la reutilización del código y hace el programa más flexible.
La sobrecarga de operadores permite redefinir operadores como + y += para que trabajen con
objetos. En este caso, estos operadores se utilizan para aumentar los puntos del personaje Pacman.

El flujo de información inicia en el main, donde se crean objetos y se llaman métodos. Estos métodos modifican los atributos del objeto (posición, puntos), y los cambios se reflejan en la salida
del programa.

Conclusiones

En esta práctica se logró comprender el uso de la sobrecarga de métodos y operadores en C++, lo cual permite que una misma función o símbolo tenga diferentes comportamientos dependiendo de cómo se utilice. A través del programa del personaje tipo Pacman, se observó cómo los métodos pueden adaptarse según los parámetros recibidos, facilitando la reutilización del código y haciendo más eficiente la programación.

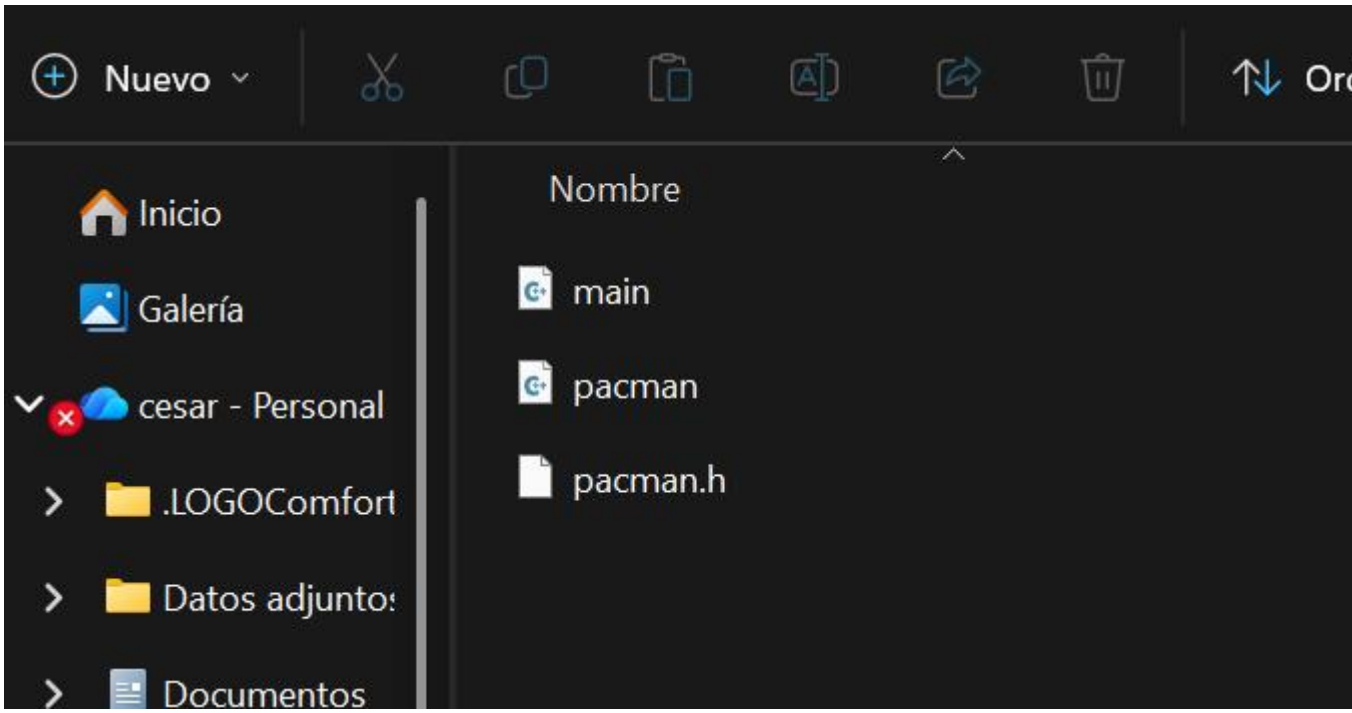
Además, se analizó la sobrecarga de operadores como + y +=, entendiendo cómo estos pueden ser redefinidos para modificar directamente el estado de un objeto, en este caso los puntos del personaje. Esto ayuda a que el código sea más intuitivo y fácil de leer.

Durante el desarrollo, se pudo identificar cómo fluye la información desde el programa principal hacia los métodos de la clase, y cómo cada acción afecta los

atributos del objeto. También se reforzó la importancia de organizar el código en diferentes archivos para una mejor estructura.

En conclusión, esta práctica permitió fortalecer los conocimientos de programación orientada a objetos, destacando la importancia de la sobrecarga como una herramienta clave para crear programas más flexibles, claros y reutilizables en situaciones reales.

Evidencia de ejecución



Aqui despues de terminar los codigos deberian aparecer en la carpeta

```
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\cesar\OneDrive\Desktop\COLLAZO\repositorio\practicas\pacman> g++ main.cpp Pacman.cpp -o programa
PS C:\Users\cesar\OneDrive\Desktop\COLLAZO\repositorio\practicas\pacman> ./progama
./progama : El término './progama' no se reconoce como nombre de un cmdlet, función, archivo de script o programa
ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta
es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ ./progama
+ ~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (./progama:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\cesar\OneDrive\Desktop\COLLAZO\repositorio\practicas\pacman> ./programa
Pacman creado en posicion (0,0)
Posicion: (0,0) | Puntos: 0
Movimiento por defecto
Posicion: (1,0) | Puntos: 0
Movimento con direccion
Posicion: (2,0) | Puntos: 0
Movimiento multiple
Posicion: (5,0) | Puntos: 0
Posicion: (5,0) | Puntos: 10
Posicion: (5,0) | Puntos: 10
PS C:\Users\cesar\OneDrive\Desktop\COLLAZO\repositorio\practicas\pacman>
```

Aqui como se observa se pudo ejecutar el codigo correctamente